FH University of Applied Sciences TECHNIKUM WIEN

Sequence Alignment- Assignment

MME – Informatics of Biological Systems

Dr Prabath Jayathissa

Student : Farah Ismail

1 Introduction

Sequence alignment is a central task in bioinformatics because it is used to compare DNA, RNA, and protein sequences in order to identify similarity, conserved regions, and evolutionary relationships. Pairwise alignment compares two sequences, while multiple sequence alignment compares a family of sequences at the same time. Exact dynamic programming methods are important because they provide optimal solutions under a scoring scheme, but they become computationally expensive as sequence length increases.

The aim of this project was to implement the required alignment methods and compare them with respect to correctness, efficiency, and biological usefulness. The project includes exact pairwise methods, affine-gap and space-efficient variants, heuristic methods, and multiple sequence alignment approaches.

2 Methods

2.1 Needleman–Wunsch

Needleman–Wunsch is a global alignment algorithm based on dynamic programming. It computes the optimal alignment over the full lengths of two sequences under a match, mismatch, and gap scoring model [1].

2.2 Smith–Waterman

Smith–Waterman is a local alignment algorithm. Instead of forcing an end-to-end alignment, it identifies the highest-scoring local region shared by two sequences [2].

2.3 Gotoh

Gotoh extended dynamic programming alignment to support affine gap penalties. In this model, opening a gap is penalized more strongly than extending an already existing gap, which is biologically more realistic than using one constant gap cost [3].

2.4 Hirschberg

Hirschberg's algorithm is a linear-space divide-and-conquer method for global alignment. It preserves the global alignment objective while reducing memory usage from quadratic to linear space [4].

2.5 BLAST-style Seed-and-Extend

BLAST uses short exact matches, called seeds, and extends them into longer local alignments. This makes it much faster than computing a full local alignment matrix, although it does not guarantee the optimal local solution [5].

2.6 Minimizer-Based Approximate Alignment

Minimizers are representative sampled k-mers selected from sliding windows. They provide anchors that can be used for approximate matching and help reduce storage and computation in long-sequence comparison [10].

2.7 Progressive Multiple Sequence Alignment

Progressive alignment methods align the most similar sequences first and then progressively build larger alignments. CLUSTAL W is a classical example of this strategy [6].

2.8 Iterative Refinement

Iterative refinement improves an initial multiple sequence alignment by repeatedly realigning parts of the alignment. MUSCLE is a standard example of combining progressive alignment with refinement steps [7].

2.9 Profile HMM Alignment

Profile hidden Markov models are probabilistic models for sequence families. They represent position-specific conservation and are widely used for sequence-to-family alignment and remote homology detection [9, 8].

3 Implementation

All methods were implemented in Python in a compact modular structure. The pairwise alignment methods were grouped together because they share similar scoring and traceback logic, while the multiple sequence alignment methods were grouped in a second module. A shared utility file was used for scoring, alignment evaluation, consensus construction, and sum-of-pairs scoring.

Needleman–Wunsch, Smith–Waterman, Gotoh, and Hirschberg were implemented using dynamic programming. The BLAST-style method used seed detection and extension. The minimizer-based method used minimizer anchors and greedy chaining. Progressive multiple sequence alignment was implemented in a simplified CLUSTAL W-style form, iterative refinement in a simplified MUSCLE-style form, and profile HMM alignment with Viterbi decoding in a simplified educational version.

Unit tests were added to verify basic correctness, including valid traceback, equal alignment lengths, and expected return types.

4 Experimental Design

4.1 Datasets

To satisfy the assignment requirements, four dataset categories were used.

Dataset 1: Short related proteins. For closely related protein sequences, globin-family proteins were selected. Example entries include human hemoglobin subunit alpha (UniProt: P69905), human hemoglobin subunit beta (UniProt: P68871), and human myoglobin (UniProt: P02144). These sequences are suitable for testing global alignment, local alignment, and multiple sequence alignment on homologous proteins [11, 12, 13].

Dataset 2: Distant homologs. For a more difficult family, cytochrome c was selected as a compact but conserved protein example. The reviewed human cytochrome c entry is UniProt P99999. This dataset is useful for testing local similarity detection and the behavior of MSA methods on more diverged proteins [14].

Dataset 3: Long genomic sequences. For long-sequence experiments, the *Escherichia coli* K-12 MG1655 reference genome (GenBank: U00096.3) and the *Arabidopsis thaliana* chloroplast genome (RefSeq: NC_000932.1) were selected. These records are useful for evaluating runtime and memory behavior, especially for Hirschberg and heuristic methods [15, 16].

Dataset 4: Synthetic controls. Synthetic sequences were generated by introducing known SNPs and indels into ancestor sequences. This made it possible to compare whether the implemented methods recovered the expected similarity pattern. Synthetic controls were especially useful for comparing exact dynamic programming methods against heuristic approximations.

4.2 Evaluation Criteria

The methods were compared using the following criteria:

- **Accuracy:** alignment score, sequence identity, and qualitative recovery of conserved regions
- **Runtime:** wall-clock execution time
- **Memory usage:** peak memory consumption
- **Parameter sensitivity:** effect of changing match, mismatch, gap-open, gap-extension, seed size, and minimizer parameters

4.3 Benchmark Procedure

For pairwise alignment, synthetic sequence pairs of increasing length were generated and aligned with Needleman–Wunsch, Smith–Waterman, Gotoh, Hirschberg, BLAST-style seed-and-extend, and minimizer-based anchoring. Runtime and peak memory were recorded automatically by the benchmark script. For multiple sequence alignment, families of related synthetic protein-like sequences were generated and aligned using progressive alignment and iterative refinement. A simplified profile HMM was then built from the progressive MSA and used to align one test sequence back to the family.

To ensure reproducibility, a fixed random seed was used in the benchmark script, and the same default scoring parameters were used unless parameter sensitivity was being tested explicitly.

5 Results

5.1 Accuracy

The exact dynamic programming methods produced the strongest alignment quality overall. Needleman–Wunsch gave the most appropriate results for full-length homologous sequences, while Smith–Waterman was better when only a local conserved region was shared. In the synthetic benchmark, Needleman–Wunsch, Smith–Waterman, and Hirschberg all reached an identity of 0.944 for sequence length 200, which shows that the exact methods were highly consistent on this dataset. Gotoh produced a slightly different score because affine gap penalties change the gap structure, but the identity remained the same.

The BLAST-style method was effective at finding strong local similarities, but its identity value of 1.000 at length 200 refers only to the detected local segment and is therefore not directly comparable to full-length global alignment identity. The minimizer-based method was useful for approximate matching and anchoring long sequences, but it did not produce a full exact alignment.

5.2 Runtime

The runtime results followed the expected pattern. The exact dynamic programming methods became slower as sequence length increased, because they require matrix-based computation. At sequence length 200, Needleman–Wunsch required 0.1089 seconds, Smith–Waterman required 0.0481 seconds, and Gotoh required 0.3331 seconds. This shows that Gotoh was the most computationally expensive among the exact methods in this benchmark.

The heuristic methods were clearly faster. At sequence length 200, the BLAST-style method required only 0.0141 seconds, while the minimizer-based method required 0.0025 seconds. These results confirm that heuristic methods are more suitable when speed is more important than optimality.

5.3 Memory Usage

The memory measurements also matched the theoretical expectations. Standard dynamic programming methods used much more memory because they store full score matrices. For sequence length 200, Needleman–Wunsch used 1359.7 KB and Smith–Waterman used 715.5 KB. Gotoh required the most memory, 3492.1 KB, because of its additional affine-gap state matrices.

Hirschberg was the strongest exact method under memory constraints. At sequence length 200, it used only 20.2 KB while still achieving the same alignment score and identity as Needleman–Wunsch. This clearly demonstrates the practical value of linear-space alignment for longer sequences.

5.4 MSA Results

For the multiple sequence alignment experiments, progressive MSA and iterative refinement produced the same sum-of-pairs score in this synthetic example, namely

-406. However, iterative refinement required more runtime, which means that in this small benchmark it did not improve the final score but increased computational cost. The profile HMM achieved a valid Viterbi score of -101.937 and successfully aligned a test sequence back to the family model.

Method	Score	Identity	Runtime (s)	Memory (KB)
Needleman–Wunsch	341	0.944	0.1089	1359.7
Smith–Waterman	341	0.944	0.0481	715.5
Gotoh	331	0.944	0.3331	3492.1
Hirschberg	341	0.944	0.1766	20.2
BLAST-style	98	1.000	0.0141	8.1

Table 1: Benchmark results for synthetic pairwise sequences of length 200.

Method	Score	Runtime (s)	Memory (KB)
Progressive MSA	-406	0.0285	85.3
Iterative refinement	-406	0.1007	85.3
Profile HMM (Viterbi)	-101.937	0.0057	58.6

Table 2: Example MSA and profile HMM results on synthetic protein-like sequences.

5.5 Direct Comparison by Use Case

The benchmark suggests the following practical conclusions. Needleman–Wunsch is best for full-length similar sequences, Smith–Waterman is best for local motifs or conserved regions, and Gotoh is best when realistic gap handling is important. Hirschberg is the most useful exact method for long global alignments under memory limits. BLAST-style alignment is most useful for fast local similarity search, while minimizer-based alignment is best for approximate matching of long sequences. For sequence families, progressive MSA provides a strong baseline, iterative refinement can improve the alignment in some cases, and profile HMM alignment is useful when a new sequence must be aligned to an existing family model.

6 Discussion

The results show a clear trade-off between alignment quality and computational efficiency. The exact dynamic programming methods produced the most reliable alignments, which is expected because Needleman–Wunsch computes an optimal global alignment and Smith–Waterman computes an optimal local alignment under the chosen scoring scheme [1, 2]. In the benchmark, Needleman–Wunsch, Smith–Waterman, and Hirschberg reached the same identity value on the synthetic sequence of length 200, which confirms that the exact methods were consistent on this dataset.

Gotoh was especially useful because affine gap penalties model insertions and deletions more realistically than a constant gap cost [3]. In practice, this makes the

resulting alignments biologically more plausible, even if the runtime and memory usage are higher. This was also visible in the benchmark, where Gotoh required the most memory among the exact methods.

Hirschberg did not improve the alignment score compared with Needleman–Wunsch, but it greatly reduced memory usage. This matches the purpose of the algorithm, because Hirschberg is designed to preserve the global alignment objective while using linear space rather than full quadratic storage [4]. Therefore, Hirschberg is the best exact choice when long sequences must be aligned under memory constraints.

The heuristic methods were much faster than the exact methods. The BLAST-style approach was effective for detecting strong local similarities, which is consistent with the seed-and-extend idea introduced in BLAST [5]. However, its result depends strongly on seed choice, and the identity of 1.000 in the benchmark only applied to the detected local region, not to the whole sequence. The minimizer-based method was even faster and is useful for approximate long-sequence comparison because minimizers provide compact anchors rather than a full dynamic programming solution [10]. The disadvantage is that this method does not guarantee an optimal alignment.

For multiple sequence alignment, the benchmark showed that progressive MSA and iterative refinement produced the same sum-of-pairs score in the tested example, while iterative refinement required more runtime. This suggests that refinement does not always improve the result, especially on small synthetic datasets. Still, iterative methods remain important because they can correct early alignment errors in more difficult sequence families, which is one of the main ideas behind MUSCLE-style refinement [7]. The profile HMM result showed that a sequence can be aligned back to a family model in a probabilistic way, which is the key strength of profile HMMs in sequence-family analysis [8, 9].

One limitation of this project is that some advanced methods were implemented in simplified form. The BLAST-style method is a compact educational prototype rather than a full database search engine, the progressive MSA does not use a full guide tree, and the profile HMM uses simplified transition modeling. However, the implemented versions were sufficient to demonstrate the main algorithmic ideas and to support a meaningful comparison across methods.

Overall, the project shows that there is no single best alignment algorithm for all tasks. Exact dynamic programming methods are best when accuracy is the priority, Hirschberg is best when memory is limited, BLAST-style and minimizer-based methods are best when speed is the main concern, and MSA or profile HMM methods are best when the goal is to analyze sequence families rather than only a single pair of sequences.

7 Conclusion

In this project, exact, affine-gap, heuristic, and multiple sequence alignment methods were implemented and compared. The main conclusion is that exact dynamic programming methods provide the best alignment quality, while heuristic methods provide better scalability on long inputs. Gotoh improves gap modeling, Hirschberg reduces memory usage, and the multiple sequence alignment methods are useful for analyzing sequence families.

Overall, the best method depends on the biological problem, sequence length,

and computational constraints. Global alignment is best for full-length homologs, local alignment is best for conserved subsequences, heuristic methods are best when speed is the priority, and MSA methods are best for studying related sequence families.

8 References

References

[1] S. B. Needleman and C. D. Wunsch. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology*, 48(3):443–453, 1970.

[2] T. F. Smith and M. S. Waterman. Identification of common molecular subsequences. *Journal of Molecular Biology*, 147(1):195–197, 1981.

[3] O. Gotoh. An improved algorithm for matching biological sequences. *Journal of Molecular Biology*, 162(3):705–708, 1982.

[4] D. S. Hirschberg. A linear space algorithm for computing maximal common subsequences. *Communications of the ACM*, 18(6):341–343, 1975.

[5] S. F. Altschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman. Basic local alignment search tool. *Journal of Molecular Biology*, 215(3):403–410, 1990.

[6] J. D. Thompson, D. G. Higgins, and T. J. Gibson. CLUSTAL W: improving the sensitivity of progressive multiple sequence alignment through sequence weighting, position-specific gap penalties and weight matrix choice. *Nucleic Acids Research*, 22(22):4673–4680, 1994.

[7] R. C. Edgar. MUSCLE: multiple sequence alignment with high accuracy and high throughput. *Nucleic Acids Research*, 32(5):1792–1797, 2004.

[8] A. Krogh, M. Brown, I. S. Mian, K. Sjölander, and D. Haussler. Hidden Markov models in computational biology: applications to protein modeling. *Journal of Molecular Biology*, 235(5):1501–1531, 1994.

[9] S. R. Eddy. Profile hidden Markov models. *Bioinformatics*, 14(9):755–763, 1998.

[10] M. Roberts, W. Hayes, B. R. Hunt, S. M. Mount, and J. A. Yorke. Reducing storage requirements for biological sequence comparison. *Bioinformatics*, 20(18):3363–3369, 2004.

[11] UniProt Consortium. UniProtKB entry P69905: Hemoglobin subunit alpha (*Homo sapiens*). UniProt, accessed April 7, 2026.

[12] UniProt Consortium. UniProtKB entry P68871: Hemoglobin subunit beta (*Homo sapiens*). UniProt, accessed April 7, 2026.

[13] UniProt Consortium. UniProtKB entry P02144: Myoglobin (*Homo sapiens*). UniProt, accessed April 7, 2026.

[14] UniProt Consortium. UniProtKB entry P99999: Cytochrome c (*Homo sapiens*). UniProt, accessed April 7, 2026.

[15] National Center for Biotechnology Information. GenBank record U00096.3: *Escherichia coli* str. K-12 substr. MG1655, complete genome. NCBI Nucleotide, accessed April 7, 2026.

[16] National Center for Biotechnology Information. RefSeq record NC_000932.1: *Arabidopsis thaliana* chloroplast, complete genome. NCBI Nucleotide, accessed April 7, 2026.